UML – Unified Modeling Language

Septembre 2023

Objectifs du cours :

- 1. Présentation de UML et du diagramme de classe.

- 2. Se représenter concrètement les principaux concepts UML du diagramme de classes (*objet*, *interface*, *classe* …) avec des exemples en Java.

Présentation UML :

- https://en.wikipedia.org/wiki/Unified_Modeling_Language

- Le **Langage de Modélisation Unifié**, de l'anglais Unified Modeling Language (UML), est un langage de modélisation graphique à base de pictogrammes conçu comme une méthode **normalisée** de **visualisation** dans les domaines du développement logiciel et en conception **orientée objet**.

- L'UML est une synthèse de langages de modélisation objet antérieurs : Booch, OMT, OOSE.
- Principalement issu des travaux de Grady Booch, James Rumbaugh et Ivar Jacobson, UML est à présent un standard adopté par l'Object Management Group (OMG). UML 1.0 a été normalisé en janvier 1997; UML 2.0 a été adopté par l'OMG en juillet 2005. La dernière version de la spécification validée par l'OMG est UML 2.5.1 (2017).

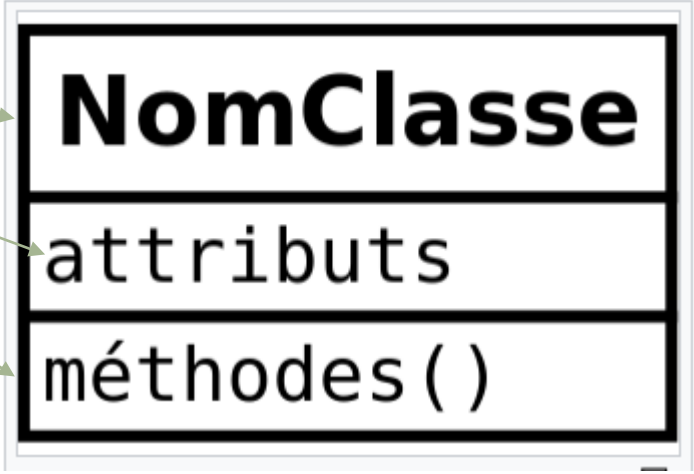
Le Diagramme de classes :

- Le **diagramme de classes** est un schéma utilisé en génie logiciel pour présenter les classes et les interfaces des systèmes ainsi que leurs relations.

- Ce diagramme fait partie de la partie statique d'UML, ne s'intéressant pas aux aspects temporels et dynamiques.

Diagramme de classes :

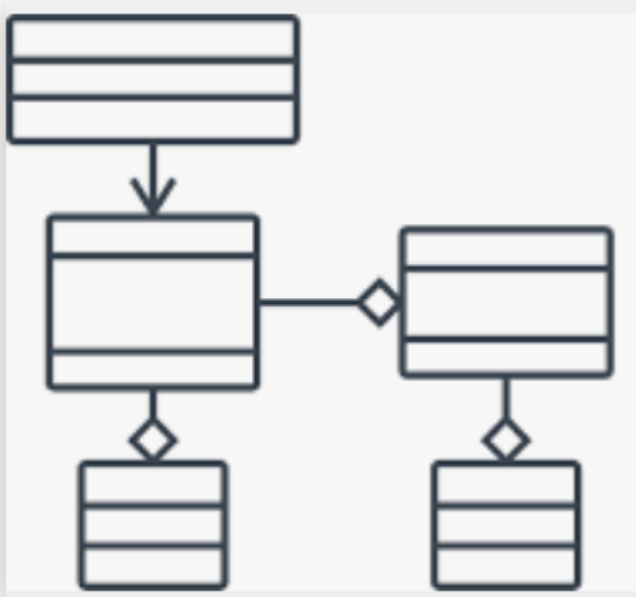
- Une classe est représentée par :
  - Son nom.
  - Ses attributs.
  - Ses méthodes.



Les relations entre classes :

- Les principales relations entre les classes

1. Interface.
2. La relation structurelle.
3. L'agrégation.
4. La composition.
5. La relation de spécialisation.
6. La relation de réalisation.
7. La relation de dépendance.

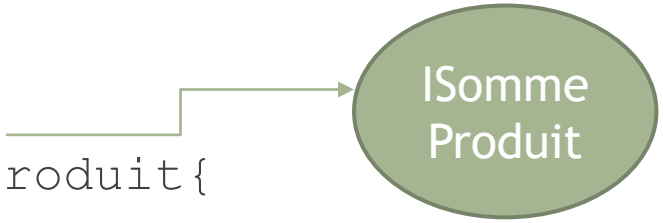


1. Une Interface :

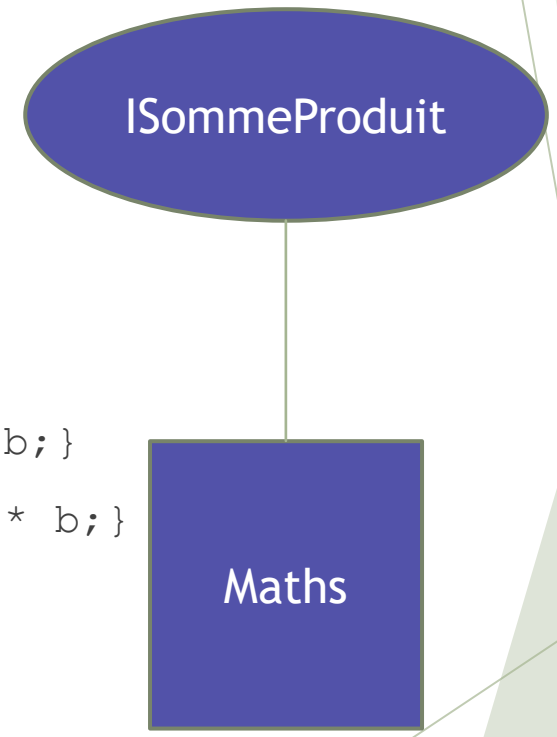
- Définir un contrat (« *de travail* »).

Exemple en Java :

- ```
  interface ISommeProduit{
  int somme();
  int produit();
  }
  ```

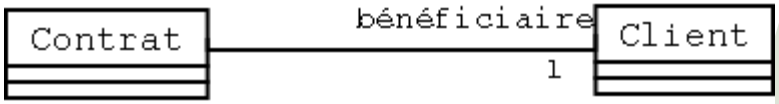


Cette interface définie le contrat c.a.d. les fonctions qui seront permises par les classes implémentant cette dernière.

Ici « *somme* » et « *produit* ».
  ```

➔ *Implémentation d'une interface :*

- ```
  class Maths implements ISommeProduit
  {
    private int a; int b;
    Maths(int a, int b) {
      this.a = a;
      this.b = b}
    public int somme(){return a + b;}
    public int produit(){return a * b;}
  }
  ```


  ```

2. La relation structurelle ou association :

- Une relation **structurelle** décrit un <u>ensemble de liens</u>.
- Un lien est une connexion entre objets.
- <u>**Exemple**</u> : Un contrat concerne un (*ou plusieurs*) clients.



- `class Contrat {`

  `Client bénéficiaire;`

  `} ou dans le cas de plusieurs clients ….`

- `class Contrat {`

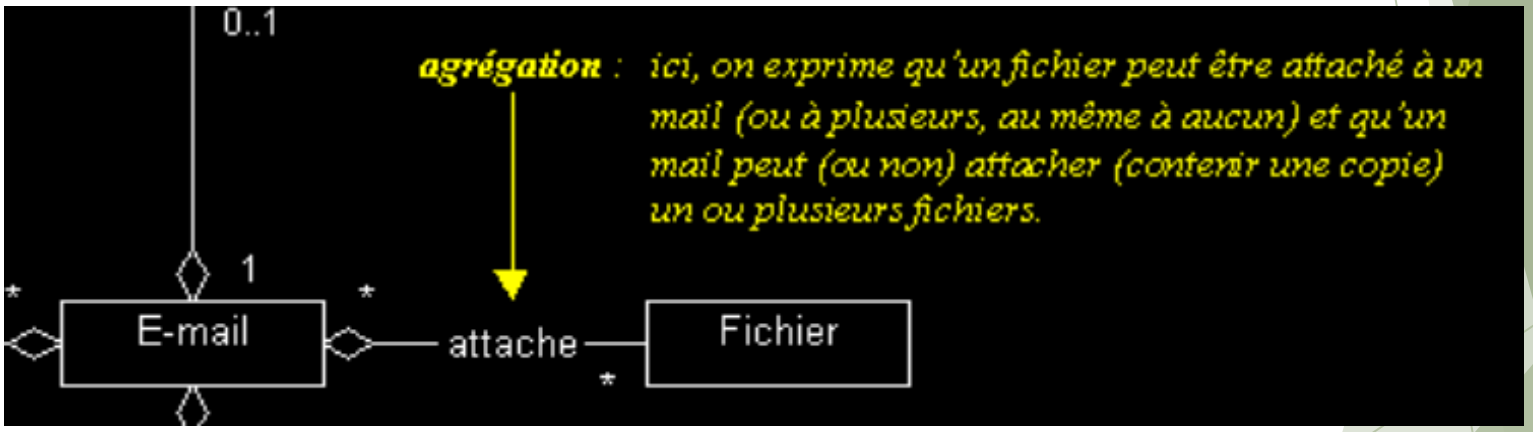
  `Client[] bénéficiaires;`

  `}`

3. L’agrégation :

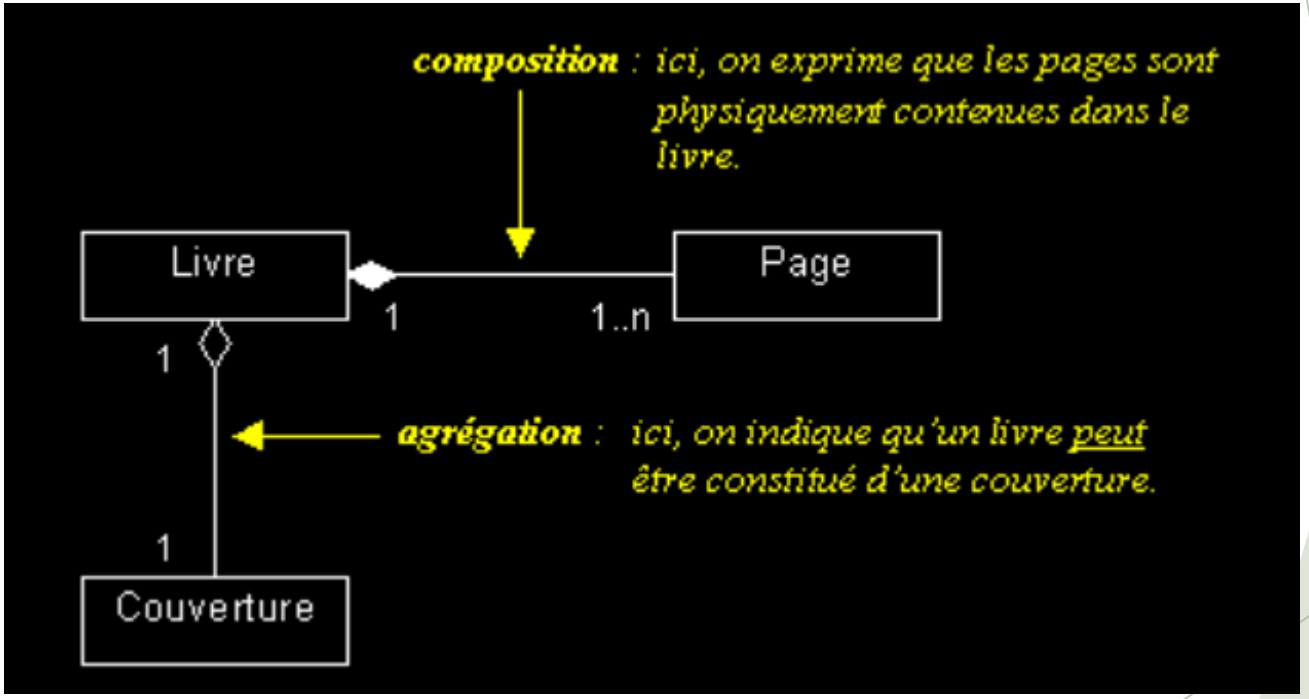
- L'agrégation est une association non symétrique, qui exprime un couplage fort et une relation de subordination.

- Elle **représente** une relation de type "**ensemble / élément**".



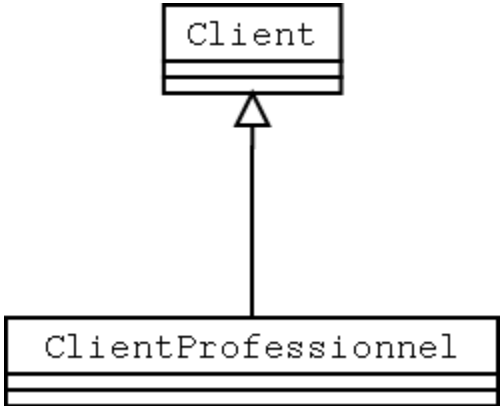
4. La composition :

- La composition est une agrégation forte (**agrégation par valeur**).



5. La relation de spécialisation :

- **Relation d'héritage**, dans laquelle les objets de l'élément spécialisé (classe enfant) peuvent remplacer les objets de l'élément général (classe parent).
- **<u>Exemple</u> :** Un client professionnel est une sorte de client.



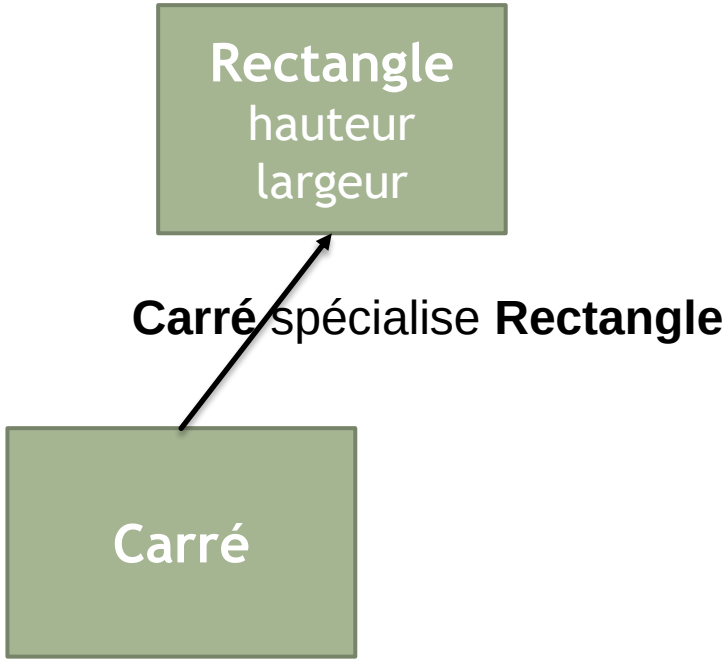
- `class ClientProfessionnel extends Client { ... }`

5. L'héritage / Généralisation :

- **Attention au principe de substitution de Liskov.**
  - Spécifie la manière dont les contrats peuvent interagir avec les mécanismes d'héritage.

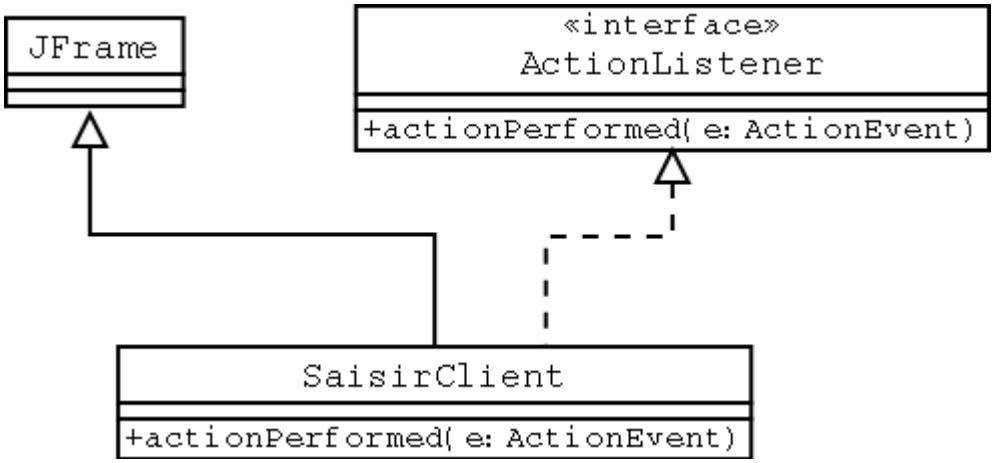
Problème :
Un carré a les deux côtés égaux … !

Réponse :
Passer par une classe générique « *Shape* ».



6. La relation de réalisation :

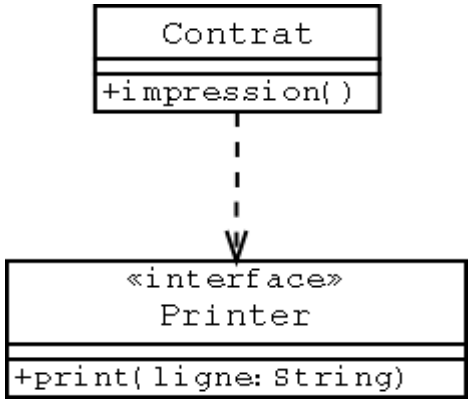
- Relation dans laquelle une interface définit un contrat garanti par une classe d'implémentation.




```
class SaisirClient extends JFrame implements ActionListener {
   ... public void actionPerformed(ActionEvent e) { // faire
   quelque chose } }
```

7. La relation de dépendance :

- Relation entre éléments du modèle ne nécessitant pas forcément un lien entre objets. Lorsque cette relation est réalisée par des liens entre objets, ces derniers sont limités dans le temps.




```
class Contrat { ...
     public void impression() {
         Printer imprimante = PrinterFactory.getInstance();
...  imprimante.print(client.getName()); ... } }
```

Exercice

Ecrire le diagramme de classes d'une bibliothèque contenant les <u>classes suivantes</u> :

Bibliothèque, Médiathèque
Adhérent
Livre, CD, DVD, Jeux
Auteur
Emprunte